

Contents

The Architecture Behind the Project	2
1. A project about more than a slider	2
2. Why deliberately make it more complex?	2
3. The project as an architectural laboratory	3
4. The central philosophical rule: ownership	4
5. Why the Slider became a hierarchy	5
6. Why the project also relies heavily on composition	5
7. Why managers matter so much	6
8. The EventManager idea	7
9. Why dynamic subscriptions exist	7
10. Why _pipe() is a key architectural idea	8
11. Why the pipe needs application-level control	9
12. Why the FSMs are local	9
13. Why no forced State pattern	10
14. The meaning of slidemove and slidechange	10
15. Why autoscroll is more than a timer	11
16. Why LOCKED is different from OFF	12
17. Why the AudioPlayer has its own state model	12
18. Why components communicate through events	13
19. The project uses patterns as vocabulary, not as decoration	13
20. Why DI and composition matter	14
21. Why the project avoids “god object” architecture	14
22. Why validation is part of the philosophy	15
23. The project as a learning progression	16
24. What was actually achieved	16
25. Why “complex but controllable” is the right description	17
26. Why the architecture is interesting beyond this project	17
27. Final Perspective	18

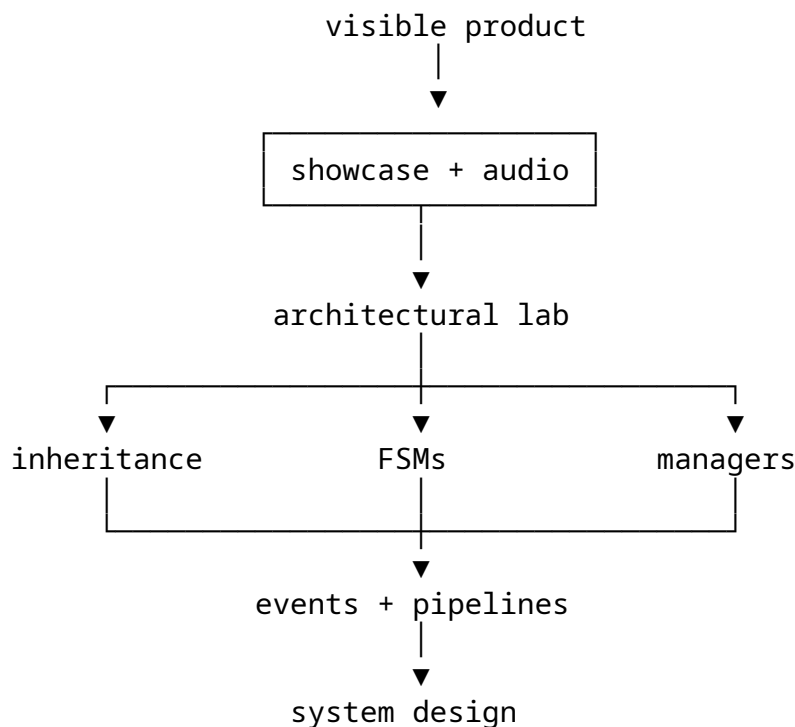
The Architecture Behind the Project

A project about more than a slider

From the outside, the project can look almost deliberately modest: an interactive showcase, a slider, some controls and an audio player. That simplicity is part of what makes it useful as an architectural experiment.

The project was not created because a slider needed an unusually sophisticated implementation. It was created because a familiar, understandable UI can serve as a laboratory for ideas that are much larger than the UI itself.

The real subject is the architecture.



The product is concrete enough to interact with, but small enough to remain mentally controllable. That makes every architectural decision visible.

1. Why deliberately make it more complex?

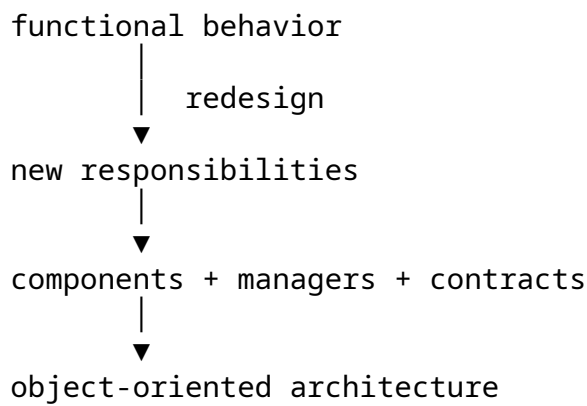
A simple slider can be implemented with a handful of functions and event listeners. That would solve the immediate UI problem.

But that was not the problem this project was intended to solve.

The real educational task was to move from a functional/procedural implementation toward an object-oriented architecture and, in particular, to explore prototype-based OOP rather than simply taking the old code and adding this everywhere.

That distinction is important.

Old implementation
↓



The project therefore became a controlled environment in which to ask questions such as:

- Where should this responsibility live?
- What should the component know about?
- What should be reusable infrastructure instead?
- What belongs in inheritance and what belongs in composition?
- Where is state local, and where does coordination belong?
- When is a pattern genuinely useful, and when is it merely decoration?

The complexity is therefore intentional. It is the material of the exercise.

2. The project as an architectural laboratory

The project was built as a portfolio and learning laboratory rather than as a production application pretending to need enterprise architecture.

That changes the measure of success.

The question is not:

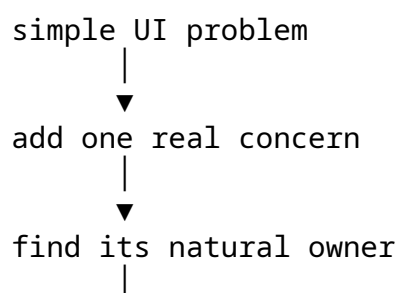
Could the same visual result be produced with fewer lines?

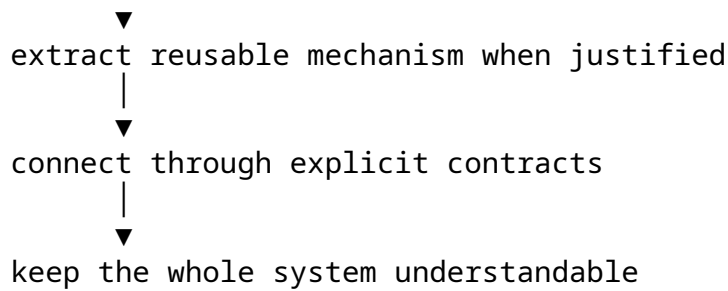
Of course it could.

The more interesting question is:

Can a deliberately richer design remain understandable, internally consistent and controllable while several independent interaction concerns coexist?

The project explores that question through a real interactive system rather than through abstract examples.





That last step matters as much as the first ones. Architecture is only useful when the developer can still reason about the result.

3. The central philosophical rule: ownership

The project can be understood through one question:

Who owns this knowledge?

The answer is deliberately different for different kinds of knowledge.

Slider
└─ owns movement and interaction state

AudioPlayer
└─ owns playback and audio state

Shop
└─ owns external link behavior

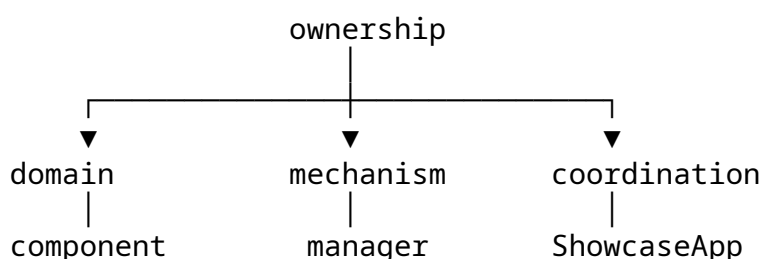
Managers
└─ own reusable mechanisms

ShowcaseApp
└─ owns application-level coordination

This is more important than any individual class name or pattern.

Once ownership is clear, many design decisions become easier.

The Slider should not know how an album is played. The AudioPlayer should not know how the slider teleports around cloned slides. A manager should not need to know what an album means. And ShowcaseApp should coordinate these things without becoming the place where all their internal algorithms are reimplemented.

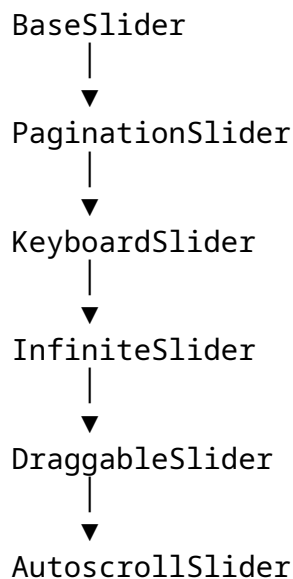


This is what keeps the architecture from collapsing into a single large object.

4. Why the Slider became a hierarchy

The slider is the most visible demonstration of the project's object-oriented thinking.

Instead of implementing one enormous slider containing navigation, pagination, keyboard input, infinite looping, dragging and autoscroll, the project grows the capability set layer by layer.



The important point is not simply that inheritance is being used. It is that each layer has a reason to exist.

The final object is still conceptually one Slider, but it has acquired capabilities progressively. That makes inheritance a tool for expressing a relationship of identity and extension:

same conceptual object
+
new capability
↓
new specialized layer

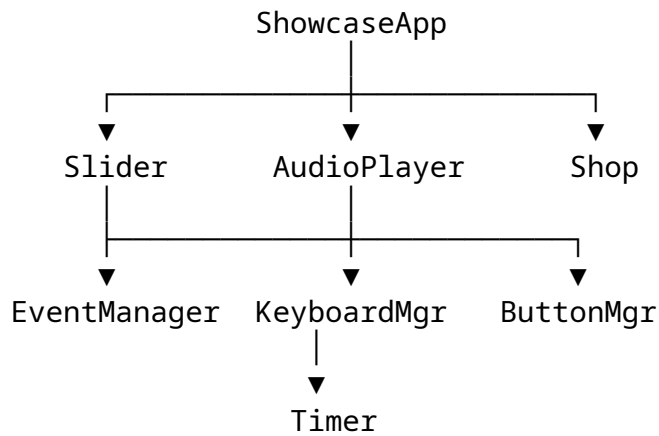
This also explains why the project was useful for learning prototypes specifically. Prototype-based JavaScript makes the inheritance machinery visible instead of hiding it behind class syntax.

5. Why the project also relies heavily on composition

Inheritance alone would not solve the architecture.

Reusable infrastructure does not belong in the Slider hierarchy. An `EventManager` is not a special kind of `Slider`. A `Timer` is not a special kind of `AudioPlayer`.

These concerns are therefore composed into the objects that need them.



The architecture consequently uses both major OOP mechanisms for different reasons:

Inheritance

→ specialize an existing domain object

Composition

→ give a component reusable capabilities/mechanisms

The project is not trying to prove that one mechanism is universally better than the other. It is learning where each one fits.

6. Why managers matter so much

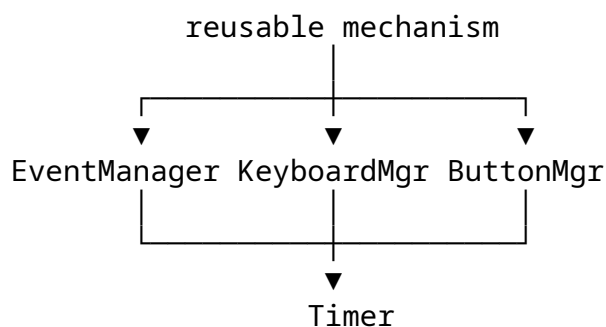
The managers are some of the least visible parts of the UI and some of the most important parts of the architecture.

Without managers, every component would gradually accumulate generic browser mechanics:

```

addEventListener()
removeEventListener()
key matching
button lookup
handler validation
timer bookkeeping
  
```

The project instead asks those mechanisms to become reusable infrastructure.



The architectural payoff is not merely code reuse.
It changes the language of the components themselves.
A component can say:

"perform next"

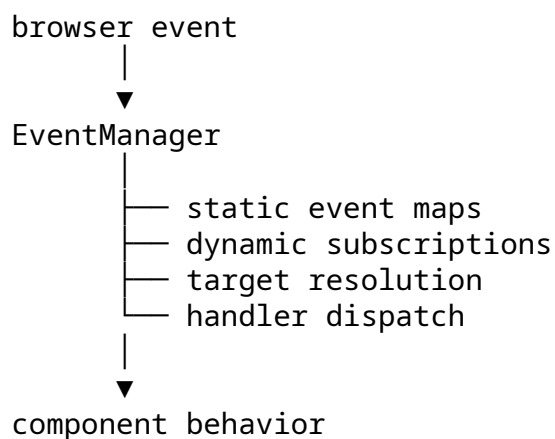
instead of:

"inspect this KeyboardEvent, compare keys, find a method,
validate it and then invoke it"

The mechanism is centralized. The domain meaning stays local.

7. The EventManager idea

The EventManager is particularly significant because it creates an infrastructure boundary between browser events and component logic.



This means event wiring becomes declarative rather than being repeated as unrelated imperative setup code.

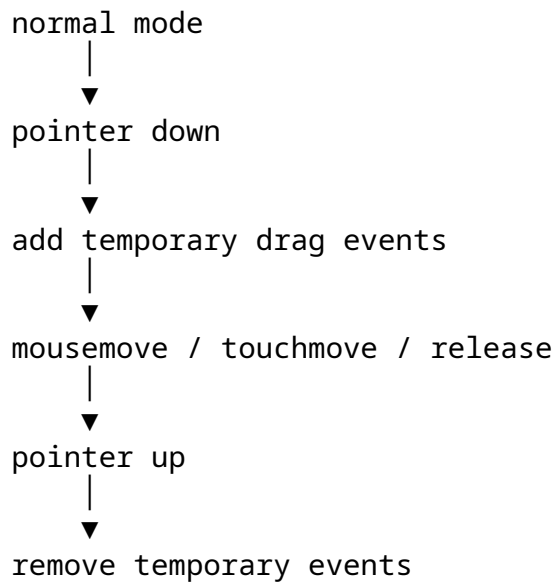
The prototype hierarchy makes this even more interesting: event maps can belong to individual capability layers, while the manager walks the chain and builds the effective subscription set.

That is an architectural expression of the same principle used in the Slider hierarchy:

let each layer own the part of the problem that belongs to that layer.

8. Why dynamic subscriptions exist

Not every event is a permanent feature of the application.
Dragging is the obvious example.



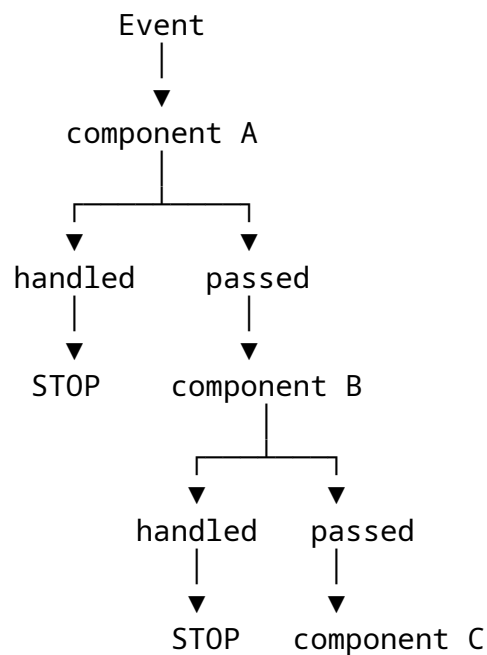
This is a small detail with a larger architectural lesson: interaction has a lifecycle.

The project therefore treats event lifetime as part of the design instead of assuming every event subscription is eternal.

9. Why `_pipe()` is a key architectural idea

The pipeline is where several otherwise separate ideas start to work together.

A browser event can be offered to several components in sequence.



The project uses the original Event as a lightweight continuation token. A handler that consumes the input can return `true`; a handler that does not consume it can preserve the original event so that another participant can try.

`true`

└─ "I handled this."

original event

└─ "I did not handle this. Continue."

This is why `_pipe()` is more than a convenience loop. It is a small protocol for cooperation. It resembles Chain of Responsibility, but without forcing a textbook handler hierarchy onto the application.

10. Why the pipe needs application-level control

The same event does not always belong to the same component first.

In ordinary interaction the pipeline can favor the Slider. In a special audio interaction it can be reversed.

NORMAL

Slider → AudioPlayer → Shop

REVERSE

AudioPlayer → Slider → Shop

This is a good example of why a pipeline becomes an architectural mechanism rather than just a helper function.

The application can change the order without rewriting the components.

ShowcaseApp therefore decides the routing policy, while the components still decide what handling the event means.

policy	→ ShowcaseApp
routing	→ pipeline
meaning	→ component

That separation is one of the project's cleanest architectural boundaries.

11. Why the FSMs are local

The project contains several kinds of state, but they do not belong to one conceptual machine.

Slider

└─ IDLE / MOVING

AutoscrollSlider

└─ OFF / ON / LOCKED

AudioPlayer

└─ IDLE / THEME / ALBUM / ALBUMTHEME

These state machines interact, but they do not own each other.

```
Slider state      → movement lifecycle
Autoscroll state → automatic-motion lifecycle
Audio state       → playback context
```

A global state machine would therefore have to describe combinations such as:

```
MOVING + LOCKED + ALBUM
IDLE + ON + THEME
...
```

That quickly becomes a matrix of unrelated concerns.

The local-FSM approach keeps each state model close to the code that understands it.

12. Why no forced State pattern

This is also an example of the project's broader pattern philosophy.

The project uses explicit FSMs, but it does not introduce a large State-pattern hierarchy simply because "state" exists.

```
real problem
  ↓
needs state modeling
  ↓
local FSM is enough
  ↓
no extra abstraction
```

A State pattern would become useful if state-specific behavior grew enough to justify separate state objects.

Until then, a compact local FSM is clearer.

The principle is simple:

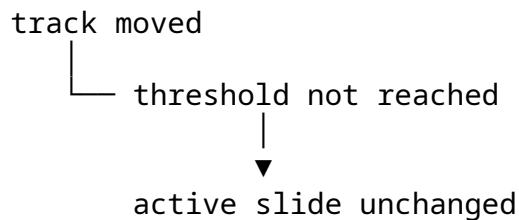
Architecture should answer a problem first; the pattern name comes second.

13. The meaning of slidemove and slidechange

One of the most valuable lessons hidden inside the implementation is the difference between physical movement and semantic change.

A track can move without the application having changed the active slide.

```
small drag
  |
  ▼
```



Therefore the application needs two different concepts:

`slidemove`
= movement started / is occurring

`slidechange`
= active slide has actually changed

That distinction is not merely implementation trivia. It protects synchronization between the slider, audio album and shop link.

A few pixels of drag should not switch the album.

This is a good example of how architectural clarity can expose and fix a subtle behavioral bug.

14. Why autoscroll is more than a timer

At first glance autoscroll sounds trivial:

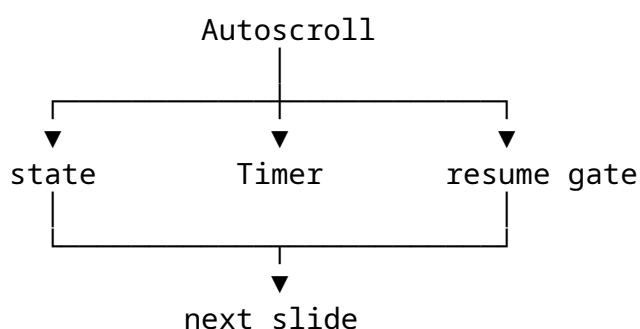
```
setInterval(next)
```

The real interaction problem is more complicated.

Autoscroll can be affected by:

- manual interaction
- hover/focus pauses
- dragging
- tab visibility
- audio playback
- explicit enable/disable
- transitions
- timing of resumption

The project therefore models autoscroll as its own subsystem.



This is why the LOCKED state exists and why `_tryResumeAutoscroll()` is meaningful: resuming is a decision, not merely a method call.

15. Why LOCKED is different from OFF

The distinction is conceptual.

OFF

= the feature is intentionally disabled

LOCKED

= the feature exists, but another condition temporarily prevents it

Album playback is the clearest example.

```
album starts
  |
  ▼
lock autoscroll
  |
  ▼
play album without competing motion
```

When the album pauses or ends, the application can decide whether autoscroll may resume.

A boolean would blur these two meanings.

The state model keeps them distinct.

16. Why the AudioPlayer has its own state model

The native `<audio>` element already has state, but it does not know the application's semantic context.

A browser property such as `audio.paused` cannot answer all application-level questions.

For example:

paused main theme

and:

paused album

are both "paused" from the browser's perspective but not from the application's perspective.

The AudioPlayer therefore introduces its own semantic mode model.

```
native media state
  +
application playback context
  ↓
AudioPlayer FSM
```

This is another instance of a general principle used throughout the project:

framework/browser state is not always the same thing as domain state.

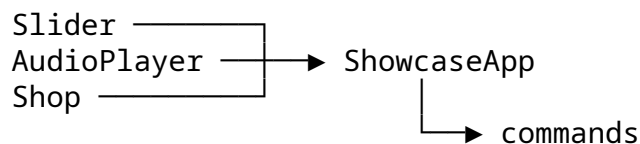
17. Why components communicate through events

The architecture deliberately avoids a web of direct dependencies.

Bad direction:



Preferred direction:



The component reports:

"something meaningful happened"

The application decides:

"what should happen elsewhere because of it"

This is the practical reason the project has an event-driven architecture.

18. The project uses patterns as vocabulary, not as decoration

The architecture has recognizable patterns, but the project does not treat them as trophies.

Mediator-like

→ ShowcaseApp

Observer / Pub-Sub-like

→ EventManager + custom events

Chain of Responsibility-like

→ `_pipe` result protocol

Strategy-like

→ configurable managers

Template Method-like

→ inheritance hooks in Slider

Command-like

→ Button

FSM

→ local state models

Some concepts are better treated as principles:

Dependency Injection

Inversion of Control

Composition

Fail-fast validation

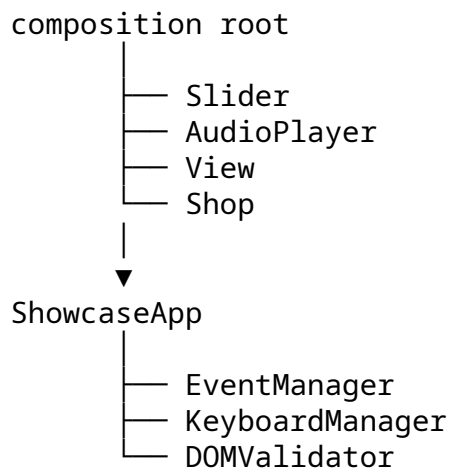
Component encapsulation

Configuration-driven behavior

The distinction matters because an architecture becomes clearer when its vocabulary is honest.

19. Why DI and composition matter

Dependencies are assembled from above rather than being discovered globally.



This keeps object construction and application policy close to the composition boundary.

It also makes the system easier to reason about: when reading the composition root, a developer can see what the application is made of.

20. Why the project avoids “god object” architecture

The easiest way to build this kind of application would have been to put everything into ShowcaseApp.

Then it could have controlled:

DOM
slider indices
transitions
dragging
autoscroll
audio
track selection
shop links
keyboard shortcuts

That would work.

It would also create an object that knew too much.

Instead:

ShowcaseApp
→ coordinates

Slider
→ owns slider mechanics

AudioPlayer
→ owns playback mechanics

Managers
→ own reusable mechanisms

This keeps the application coordinator broad in responsibility but shallow in domain knowledge.

21. Why validation is part of the philosophy

The project prefers explicit contracts over silent guessing.

If the component requires an element, it should exist.

If the configuration refers to a handler, the handler should exist.

If a state token is invalid, the setter should reject it.

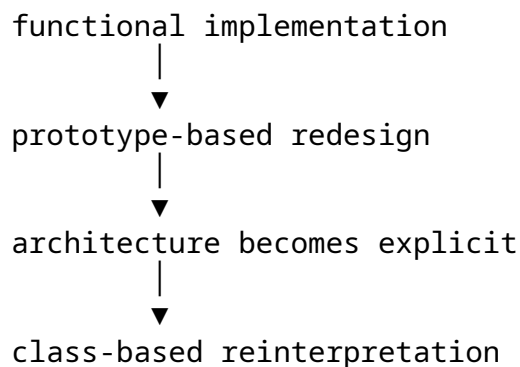
```
assumption
  |
  ▼
validate at boundary
  |
  ├── valid → continue
  └── invalid → fail early
```

This is a design philosophy as much as an implementation technique.

A hidden fallback can make an architectural mistake harder to see. A fail-fast contract exposes it where it was introduced.

22. The project as a learning progression

The architecture also represents a personal progression in how the same idea is approached.



The important part of this progression is not syntax.

It is the increasing ability to reason about:

responsibility
boundaries
contracts
state
composition
polymorphism
reusability

The second implementation therefore serves as a comparison point: the architecture can survive a change of object-oriented syntax.

23. What was actually achieved

The finished result goes beyond “a slider with features”.

It demonstrates that the original idea could be decomposed into:

independent domain components
+
reusable infrastructure
+
local state machines
+
polymorphic capability layers
+
declarative interaction configuration
+
application-level coordination

The architecture also proved useful during debugging and evolution.

Subtle problems became easier to locate because responsibilities were visible. A bug in slide-change semantics could be traced to the movement/commit boundary. Autoscroll behavior

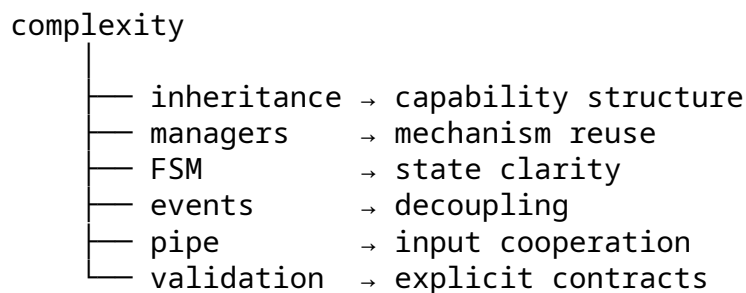
could be reasoned about through its own state and resume conditions. Keyboard propagation could be examined as a pipeline rather than as unrelated handlers.

That is a meaningful result of architectural work: architecture is not only something you document after implementation. It becomes a tool for finding and explaining behavior during implementation.

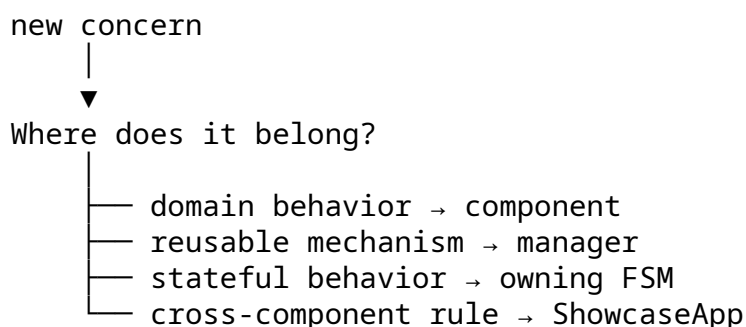
24. Why “complex but controllable” is the right description

The project is intentionally more complex than necessary for the visual problem.

But the complexity is organized.



The architecture succeeds when a new behavior can be mentally placed somewhere.



That is the real measure of control.

25. Why the architecture is interesting beyond this project

The particular UI is small, but the architectural questions are not.

A larger application will still need to decide:

who owns state?
who owns infrastructure?
who coordinates components?
how do events propagate?
what should be reusable?
where does polymorphism belong?
when is abstraction justified?
when is a pattern unnecessary?

The project therefore acts as a compact model of problems that occur at larger scales. The advantage of doing this in a small, understandable system is that cause and effect remain visible.

26. Final Perspective

The most important thing achieved here is not the number of classes, managers or patterns. It is the shift in thinking from:

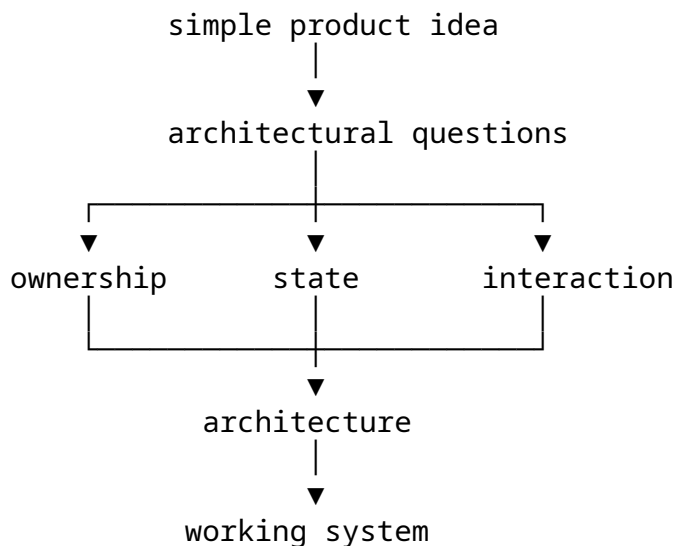
"How do I make this feature work?"

to:

"What is the responsibility behind this feature?
Who should own it?
What should the component know?
What mechanism can be reused?
What state is actually being modeled?
How should the rest of the system learn that something happened?"

That is why the project became larger than the original UI idea.

The application is the visible result. The architectural reasoning is the real subject.



And that is the philosophy of the project in one sentence:

Build enough architecture to make the system understandable, extensible and controllable - but only where the problem genuinely gives that architecture a reason to exist.

This is not a production framework disguised as a slider. It is a deliberate laboratory for learning how a real interactive system can be designed from responsibilities, state, contracts and cooperation rather than from features alone.